

Cumhuriyet Üniversitesi

İşletim Sistemleri Dersi

Bölüm 6

CPU Planlama

(CPU Scheduling)

Dr. Halil ARSLAN

Bölümün hedefleri

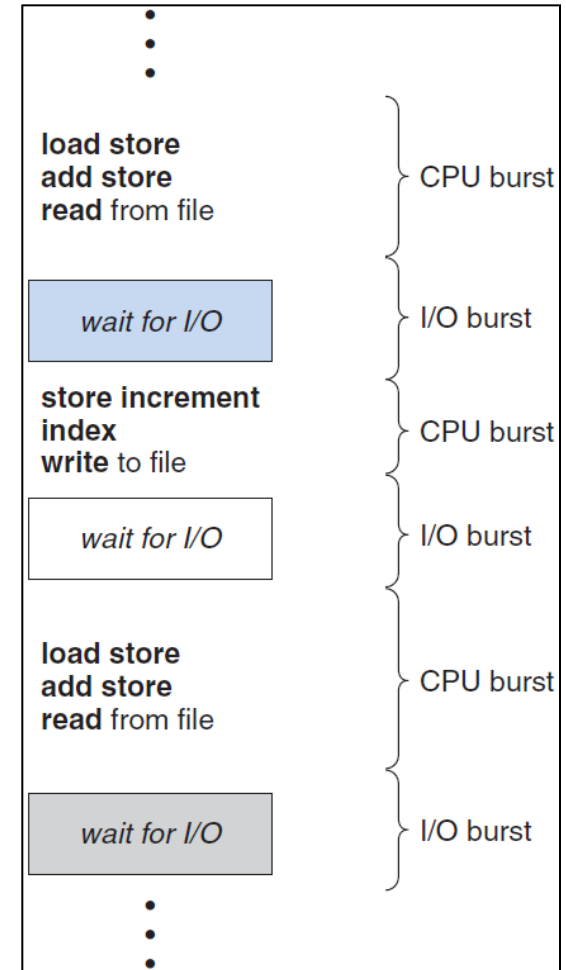
- CPU planlama
- Planlama Kriterleri
- Planlama Algoritmaları
- Thread Planlama
- Multiprocessor Planlama
- Gerçek Zamanlı CPU Planlama
- İşletim Sistemleri Örnekleri
- Algoritma Değerlendirme

CPU Planlama

- Tek işlemcili bir sistemde aynı anda sadece bir tane süreç çalışır. Diğer süreçler CPU boşa düşünceye ve yeniden planlanıncaya kadar bekletilir,
- Ancak CPU'da yürütülen bir süreç genellikle G/Ç isteğinde bulunduğu bekleme durumuna düşer ve bu sürede CPU boşa bekler,
- Multiprogramming sistemlerde ise bazı süreçlerin her zaman çalışmasının sağlanmasının yanında CPU'nun boşa beklemesini en aza indirerek sistemin verimliliğinin artırılması istenir,
- Bu nedenle **CPU Planlama (CPU scheduling)**, multiprogrammed işletim sistemlerinin temelini oluşturur.
- Çünkü multiprogrammed işletim sistemleri, bilgisayarın daha verimli çalışmasını sağlamak için CPU'yu süreçler arasında paylaştırarak kullanır.
- Bu anlamda planlama kavramı, bir işletim sisteminin temel fonksiyonu olmakla birlikte, en önemli kaynak olan CPU'nun planlanması işletim sisteminin merkezinde yer alır.

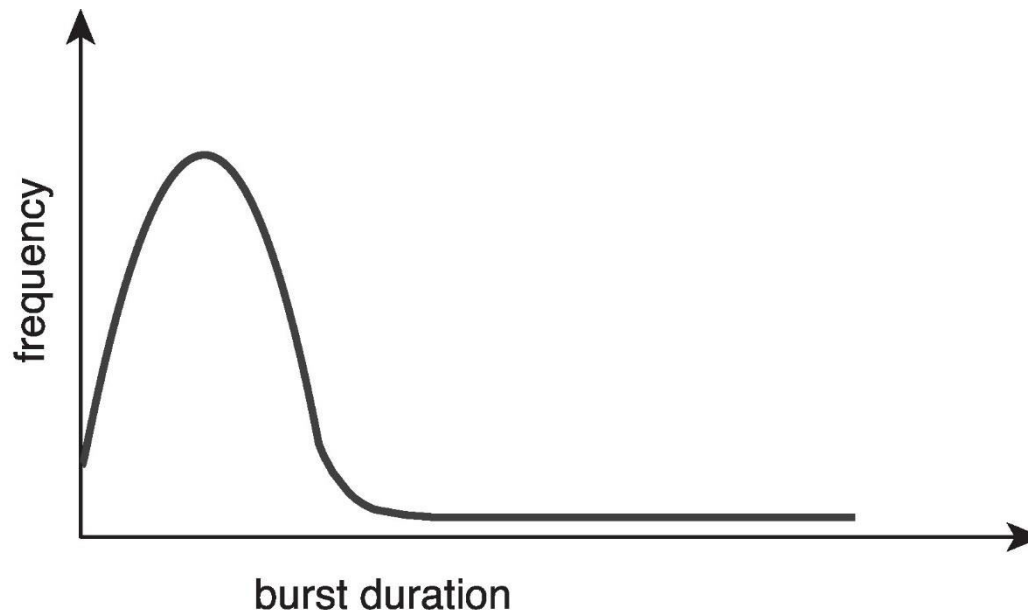
CPU Planlama

- Süreçlerin yürütümünde CPU planlama başarımı, süreçlerin CPU'da yürütümü (**CPU burst**) ve G/Ç aygıtlarında bekleme (**I/O burst**) koşullarına bağlıdır.
- Bir süreç CPU burst ile başlar sonrasında I/O burst ile devam eder ve sonunda bir sistem çağrısı olan CPU burst ile sonlanır.
- G/Ç bağımlı bir programın genellikle çok sayıda ama kısa CPU burst'leri, CPU bağımlı programlarınsa birkaç uzun CPU burst'leri olur.
- Bu dağılım uygun bir CPU planlama algoritması seçiminde önemli parametreler sunar.



CPU Planlama

- **CPU burst** süreleri, süreçten sürece ve bilgisayardan bilgisayara büyük farklılıklar gösterebilseler de genellikle aşağıdaki eğriye uygun davranış göstermektedir.
- Eğri genellikle az sayıda uzun CPU burst, çok sayıda kısa CPU burst sürelerine sahiptir.



CPU Planlama

- CPU boşa düştüğünde, işletim sistemi hazır kuyruktaki (**ready queue**) süreçlerden birini seçer,
- Bu seçim kriteri, **short-term planlama** (kısa aralıklarla **context switching**) yada CPU planlama algortimasına bağlı olarak yapılabilir,
- Hazır süreç kuyruğundan seçilen sürece CPU tahsis edilir.
- Hazır süreç veri yapısı FIFO kuyruğu olabileceği gibi planlama algoritmasına bağlı olarak priority kuyruğu, bir ağaç yapısı veya basit bir sırasız liste de olabilir,
- Ancak kavramsal olarak hazır süreç kuyruğunda bekleyen tüm süreçler CPU'yu ele geçirme şansına sahip olabilecek şekilde sıralanırlar,
- Hazır süreç kuyruğunda bekleyen süreçler süreç kontrol blokları (PCBs) olarak tutulmaktadır.

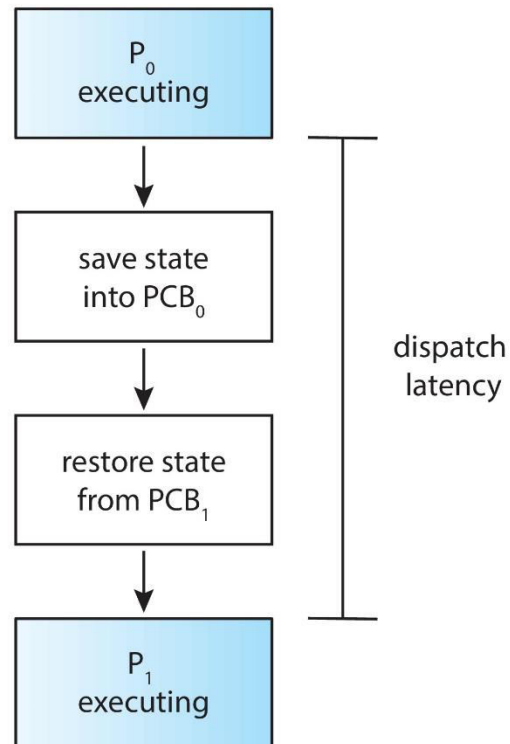
CPU Planlama

- CPU planlama, aşağıdaki 4 durum meydana geldiğinde gerçekleşir.
 1. Bir süreç çalışma (**running**) durumundan bekleme (**waiting**) durumuna geçtiğinde. (bir G/Ç isteğinde yada bir child sürecin sonlanmasının beklenildiğinde wait() durumu)
 2. Bir süreç çalışma (**running**) durumunda hazır (**ready**) durumuna geçtiğinde (bir kesme oluştuğunda)
 3. Bir süreç bekleme (**waiting**) durumunda hazır (**ready**) duruma geçtiğinde (bir G/Ç isteğinin tamamlanmasında)
 4. Süreç sonlandığında
- 1. ve 4. durumlar için planlama zorunludur. Çünkü çalıştırılmak üzere başka bir süreç seçilmelidir. Bu planlamaya **nonpreemptive** (kesintisiz) veya **cooperative** planlama denir. Diğer durumlar **preemptive**'dir.
- **Nonpreemptive** planlamada CPU'nun tahsis edildiği süreç sonlanıncaya kadar yada beklemeye geçinceye kadar CPU tutulur.

CPU Planlama

- CPU planlama işlevinde, short-term planlama ile seçilen sürecin CPU'nun kontrolüne verilmesini sağlayan modül **dispatcher** olarak isimlendirilir.
- Bu modül;
 1. İçerik değişimi (Switching context)
 2. Kullanıcı moduna geçiş
 3. Değişim yapılan kullanıcı programını sürdürmek için uygun konuma dallanma işlevlerini yerine getirir
- Her bir süreç değişimi durumunda dispatcher mümkün olduğunca hızlı olmalıdır.
- Dispatcher'in bir süreci durdurması ve diğer süreci çalıştırması için geçen zaman **dispatch latency** (dağıtım gecikmesi) olarak isimlendirilir.

CPU Planlama



Planlama Kriterleri

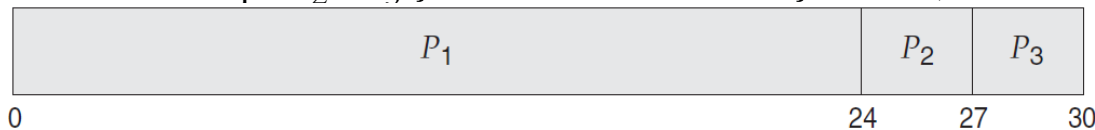
- CPU planlama algoritmalarını karşılaştırmak için bazı kriter kullanılır.
 - **CPU utilization:** CPU mümkün olduğunca kullanımda olmalıdır. Teoride CPU %0-%100 arasında olsa da gerçek ortamda %40-%90 arasındadır.
 - **Throughput** (iş hacmi): Tanımlanan bir zamanda tamamlanan süreç sayısını ifade eder.
 - **Turnaround time**(Dönüş süresi): Bir sürecin belleğe yüklenmesinden tamamlanıncaya kadar geçen zamanı tanımlar. Dönüş süresi, belleğe yüklenme, hazır kuyrukta bekleme, CPU'da çalışma ve G/Ç işlemlerinde harcanan zamanların toplamıdır.
 - **Waiting time:** Bekleme zamanı bir sürecin hazır kuyrukta geçirdiği zamanı tanımlar. Çünkü CPU planlama algoritmaları bir işin G/Ç ve CPU yürütüm zamanlarını etkilemez.
 - **Response time:** Bir sürecin hazır kuyruğa alındıktan sonra ilk olarak CPU'da işlenmeye başladığı zaman aralığıdır. Turnaround time'ın çıktı alınması gereken zamanı tanımlaması, interaktif sistemler açısından en iyi ölçüm kriteri olmamasına neden olmaktadır.

Planlama Algoritmaları

CPU planlama, CPU'nun hazır kuyrukta bekleyen süreçlerden hangisine tahsis edileceğinin belirlenmesidir.

1. First-Come, First-Served Scheduling: Bu algoritmada CPU'yu ilk talep eden süreç CPU'ya ilk sunulan süreç olur. Basir bir FIFO kuyruk yapısıdır. Yapı basittir ancak ortalama bekleme zamanı yüksektir. Örneğin P_1 , P_2 ve P_3 süreçleri için;

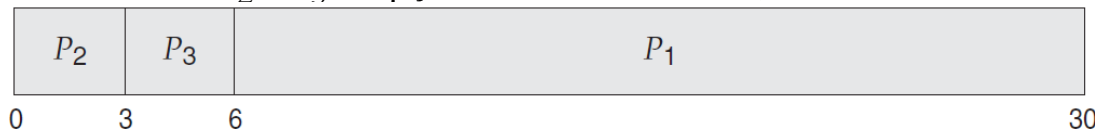
Sıralama P_1 , P_2 , P_3 şeklinde ise Gant şeması;



Waiting time $\Rightarrow P_1$ 0ms, P_2 24ms, P_3 27 ms'dir.

Ortama waiting time $\Rightarrow (0+24+27)/3 = 17$ ms'dir.

Sıralama P_2 , P_3 , P_1 şeklinde olsa idi;



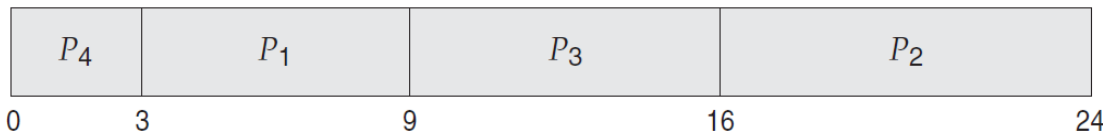
Ortama waiting time $\Rightarrow (0+3+6)/3 = 3$ ms'dir.

Process	Burst Time
P_1	24
P_2	3
P_3	3

Bu algoritma uzun bir sürecin CPU'yu elinde tutması ile kısa süreçlerin hazır kuyrukta uzun süre beklemesine neden olacağından (convoy effect) zaman paylaşımli sistemler için uygun değildir. **Nonpreemptive** bir algoritmadır.

Planlama Algoritmaları

2. Shortest-Job-First Scheduling: Bu algoritmada CPU, en kısa CPU burst zamanına sahip sürece tahsis edilir. CPU burst süreleri **eşit süreçlerde FCFS** gibi davranır. Shortest-next-CPU-burst algorithm (SJF) olarak da isimlendirilir. Örneğin P_1 , P_2 , P_3 ve P_4 süreçleri için; Gant şeması;



Process	Burst Time
P_1	6
P_2	8
P_3	7
P_4	3

Waiting time $\Rightarrow P_1$ 3ms, P_2 16ms, P_3 9 ms, P_4 0 ms'dir.

Ortama waiting time $\Rightarrow (3+16+9+0)/4 = 7$ ms'dir. Aynı iş listesi için FCFS 'de ortalama waiting time $\Rightarrow 10.25$ ms olacaktır.

Bu algoritma, belirli bir süreç kümesi için minimum ortalama bekleme süresi sağlar. Bu algortmadaki en önemli zorluk sonraki CPU isteğinin uzunluğunun belirlenmesidir. SJF algoritması toplu iş sistemlerinde long-term planlama için kullanıcının belirlediği süre sınırı baz alınarak işletilebilir.

Ancak SJF, short-term planlama sistemlerinde tercih edilmez. Çünkü sonraki CPU burst değerinin bilinmesi imkansızdır. Ancak bir önceki CPU burst sürelerine benzeterek kullanılır

Sonraki CPU burst zamanı, önceki CPU burst'lerin üssel bir ortalaması olarak hesaplanır.

Planlama Algoritmaları - SJF devam

Üssel ortalamayı, t_n , n'inci CPU burst değeri kabul edilirse, tahmin edilen (n+1)'nci CPU burst değeri τ_{n+1} olarak aşağıdaki formülle hesaplanır. τ_n , geçmiş CPU burst değerlerinin yansımasıdır. α yaygın olarak 1/2 tercih edilir.

$$\tau_{n+1} = \alpha t_n + (1 - \alpha) \tau_n \quad (0 \leq \alpha \leq 1) \Rightarrow \alpha=0.5 \text{ ise } \tau_{n+1} = 0.5(t_n + \tau_n)$$

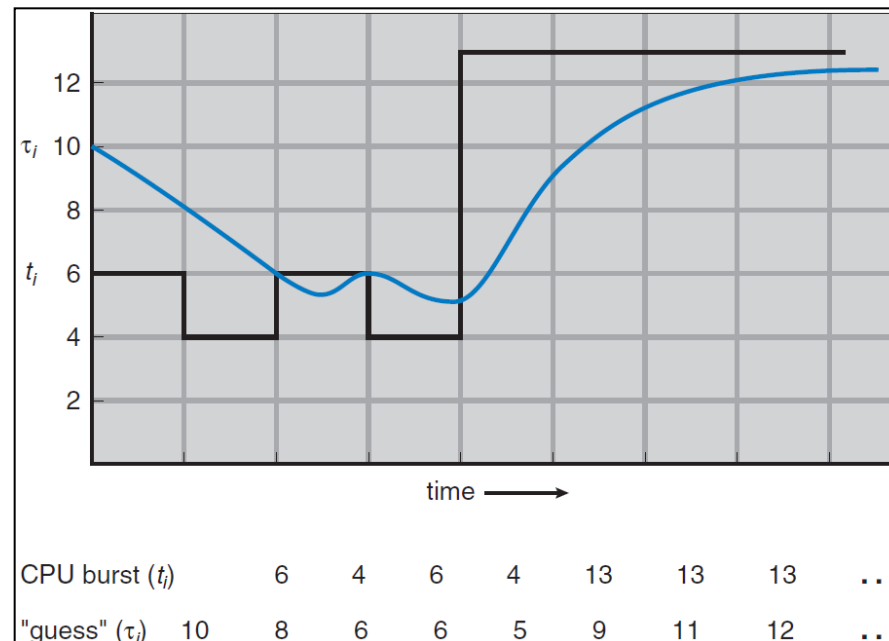
Formülün açık hali;

$$\tau_{n+1} = \alpha t_n + (1 - \alpha) \alpha t_{n-1} + \dots$$

$$+ (1 - \alpha)^j \alpha t_{n-j} + \dots$$

$$+ (1 - \alpha)^{n-1} \tau_0 \text{ 'dir. (İlk } \tau_0, \text{ bir sabit yada sistem ortalaması)}$$

**$\alpha = 1/2$ ve $\tau_0 = 10$ iken
(exponential average)**



Planlama Algoritmaları – SJF devam

- Önceki hesaplanan SJF CPU burst değerleri 6,4,6,4,13,13 şeklinde olan bir sürecin $T_0=10$ ve $\alpha=0,5$ için τ_{n+1} (τ_6) değerini hesaplayın.

$$\tau_{n+1} = \alpha t_n + (1 - \alpha) \tau_n \Rightarrow 0,5 (t_n + \tau_n)$$

$$\tau_1 = 0,5 (6+10) = 8 \text{ (ilk iş çalıştırıldıktan sonra)}$$

$$\tau_2 = 0,5 (4+8) = 6 \text{ (ikinci iş çalıştırıldıktan sonra)}$$

$$\tau_3 = 0,5 (6+6) = 6 \text{ (üçüncü iş çalıştırıldıktan sonra)}$$

$$\tau_4 = 0,5 (4+6) = 5$$

$$\tau_5 = 0,5 (13+5) = 9$$

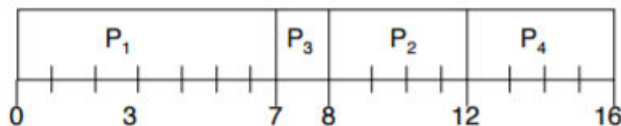
$$\tau_6 = 0,5 (13+9) = 11 \text{ (altıncı iş çalıştırıldıktan sonra)}$$

Planlama Algoritmaları – SJF devam

- SJF algoritması **preemptive** veya **nonpreemptive** olabilir.
- Yeni bir süreç hazır kuyruğa geldiğinde yeni gelen süreç o anda yürütülen sürecin kalanından daha kısa olabilir.
- Bu durumda preemptive sistemler yürütülen süreci keserken, nonpreemptive sistemler yürütülen sürecin tamamlanmasına izin verir.

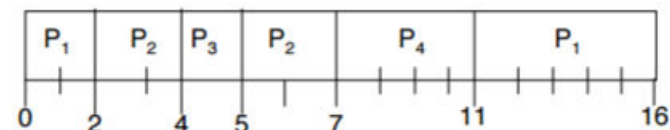
Process	Arrival Time	Burst Time
P_1	0	7
P_2	2	4
P_3	4	1
P_4	5	4

SJF (non-preemptive)



$$\text{Ort. waiting time} = (0+6+3+7)/4=4$$

SJF (preemptive)



$$\text{Ort. waiting time} = (9+1+0+2)/4=3$$

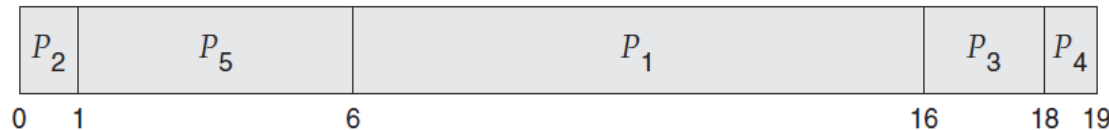
Planlama Algoritmaları

3. Priority Scheduling: Her iş bir öncelik numarası atanır ve en yüksek önceliğe sahip sürece CPU tahsis edilir.

SJF algoritmasının farklı bir varyansıdır. Bu algoritmadaki öncelikler, SJF’de beklenen CPU burst zamanının öncelik olarak kullanılması gibidir.

Eşit öncelikli süreçler, FCFS’ye göre planlanır. Önceliklerde yüksek değerlerin mi «yüksek öncelikli», düşük değerliklerin mi «yüksek öncelikli» olduğu konusunda mutabakat yoktur. Genellikle düşük değerler yüksek öncelikli alınır.

Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2



Yukarıdaki süreçler için ortalama waiting time = $(6+0+16+18+1) = 8,2$ ms’dir.
Preemptive veya nonpreemptive sistemlerde öncelik planlaması SJF gibidir.

Planlama Algoritmaları – Priority devam

- Bu algoritmadaki en önemli problem belirsiz engellenme (**indefinite blocking**) veya açlık (**starvation**) durumudur.
- Preemptive sistemlerde çalışmaya hazır düşük öncelikli bir süreç, öncelik algoritması tarafından **sonsuz dek engellenebilir**.
- Örneğin ağır yük altında çalışan bir sistemde sürekli yüksek öncelikli süreçler bir akış oluşturunca, düşük öncelikli bir sürecin CPU'ya tahsis edilmesi mümkün olmayabilir.
- Söylentilere göre, MIT'de 1973 yılında, IBM 7094 makinesi kapatıldığında, 1967'de sunulmuş ancak henüz çalıştırılmamış bir süreç bulunmuştur.
- Düşük öncelikli süreçlerin açlık durumunda kalmasının önlenmesi için yaşlandırma (**aging yada boosting**) önerilmiştir.
- Yaşlandırma, sistemde uzun süre bekleyen süreçlerin önceliğini artırmayı sağlar
- Örneğin öncelik aralığı 127 (düşük) ... 0 (yüksek) aralığındaki bir sistemde 127 önceliğe sahip bir sürecin her 15 dakikada bir önceliği azaltılır. Dolayısıyla bu süreç en kötü senaryoda 32 saat sonra en yüksek önceliğe sahip olacaktır.

Planlama Algoritmaları

4. Round-Robin (RR) Scheduling:

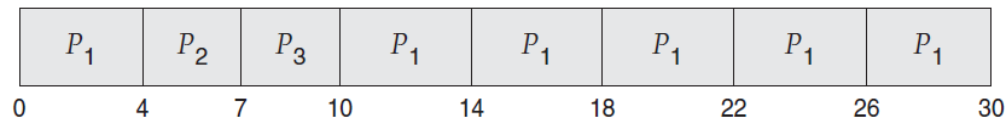
- Bu algoritma öncelikle **zaman paylaşımli sistemler** için tasarlanmıştır.
- FCFS'ye benzer fakat preemptive sistemler için süreçler arasında geçiş sağlar.
- **time quantum** veya **time slice** olarak isimlendirilen 10-100 milisaniye arası küçük zaman birimlerinde hazırda bekleyen süreçler arasında CPU paylaştırılır.
- RR zamanlayıcısı, hazır süreç kuyruğunu **FIFO yapısında** modeller. Yeni süreçler hazır kuyruğun sonuna eklenir.
- CPU planlamacı, hazır kuyruktan bir işlemi seçer ve zamanlayıcı «1 time slice» zaman sonra kesme ile sürecin hazır kuyruğa eklenmesi sağlar.
- Eğer süreç time slice'dan daha önce tamamlanırsa CPU serbest bırakılır.
- RR planlamada ortalama bekleme süresi fazladır.

Planlama Algoritmaları - RR devam

- Varış zamanları 0 olarak kabul edilen 4 sürecin CPU burst değerleri aşağıda verilmiştir.

Process	Burst Time
P_1	24
P_2	3
P_3	3

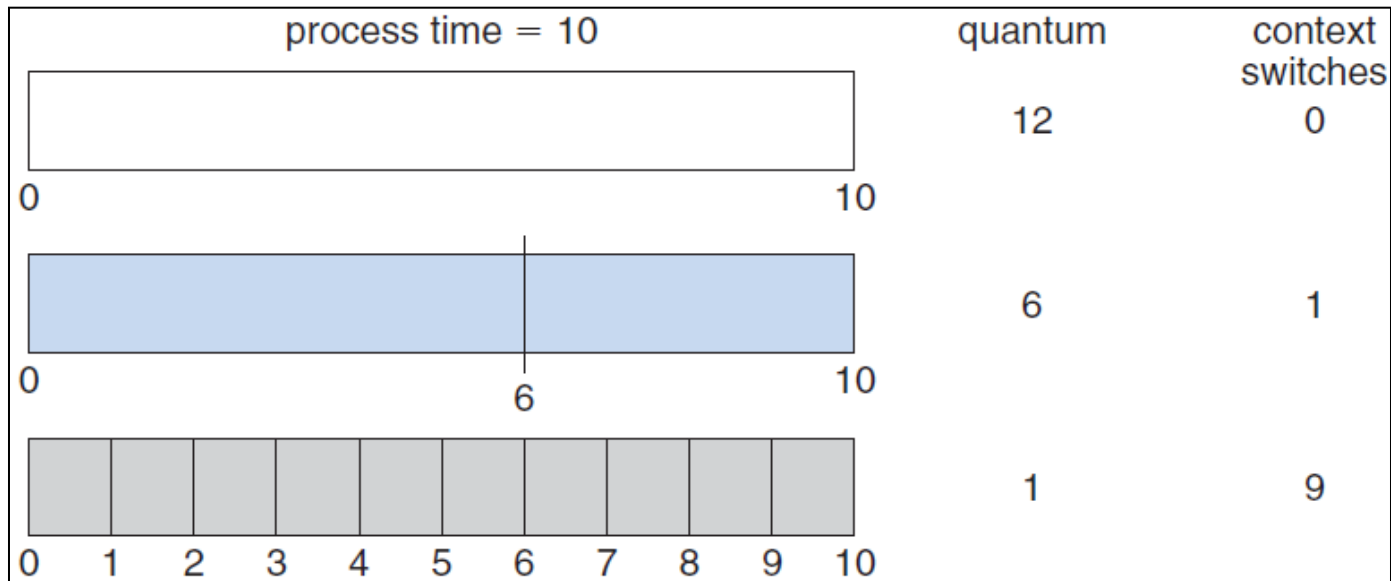
- 4 ms'lik bir time slice kullanıldığında, P_1 işlemi ilk 4 ms'de CPU'yu devralır, ardından işlemci P_2 sürecine tahsis edilir.
- P_2 süreci 4 ms dolmadan tamamlandığı için CPU'yu 3. ms'de serbest bırakır.



- Bu durumda P_1 için waiting time $(10-4)=6$, P_2 için 4 ve P_3 için 7 ms iken ortalama waiting time $= 17 / 3 = 5,66$ ms'dir.
- Bu yapısından dolayı RR algoritması **preemptive** yapıdadır.

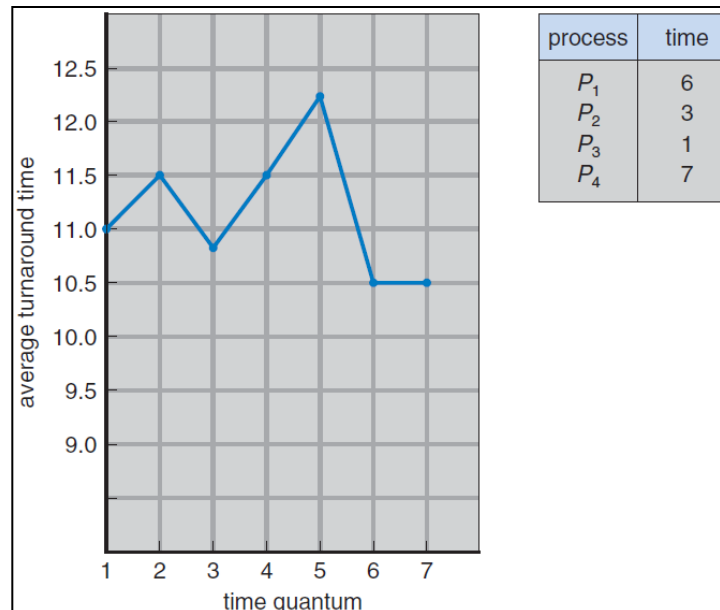
Planlama Algoritmaları - RR devam

- Bu yaklaşımda quantum (time slice) değeri algoritmanın verimliliği belirler.
- Eğer quantum çok yüksek belirlenirse FCFS gibi çalışırken, çok düşük belirlendiğin çok fazla context switching gerçekleşir ve verim düşer.
- Aşağıdaki görselde CPU burst zamanı 10 ms olan bir sürecin farklı quantum (time slice) değerlerinde gerçekleşen context switching durumları görülmektedir.
- Uygulamada genellikle quantum, context switch süresinin 10 katı alınır.



Planlama Algoritmaları - RR devam

- Turnaround time (dönüş süresi), quantum zamanına bağlıdır. Quantum zamanının artırılması ortalama dönüş süresinin iyileştirilmesi anlamına gelmez.
- Süreçlerin çoğu, tek bir quantum zamanında tamamlanırsa, ortalama dönüş süresi iyileşir.



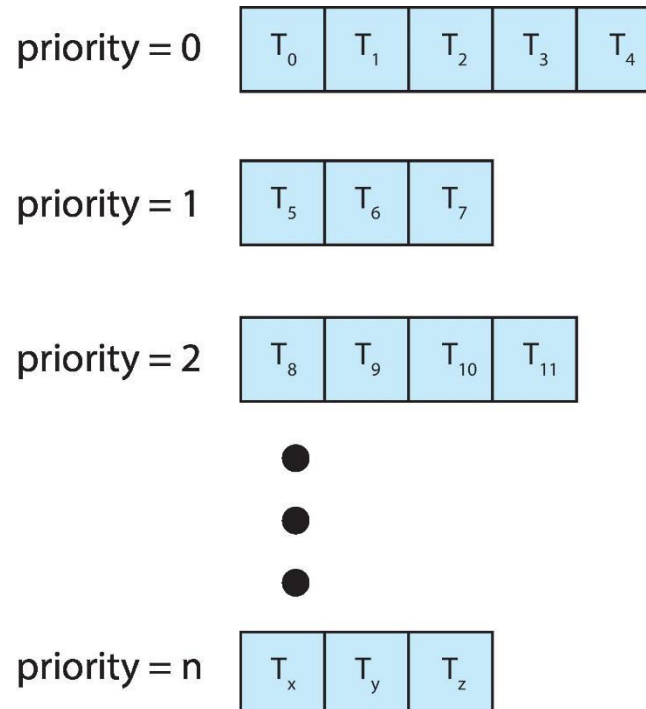
$Q=1$ için; Ortalama Turnaround Time = $(15+9+3+17)/4 = 11$ ms.

P1	P2	P3	P4	P1	P2	P4	P1	P2	P4	P1	P4	P1	P4	P1	P4	P4	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17

Planlama Algoritmaları

5. Multilevel Queue Scheduling:

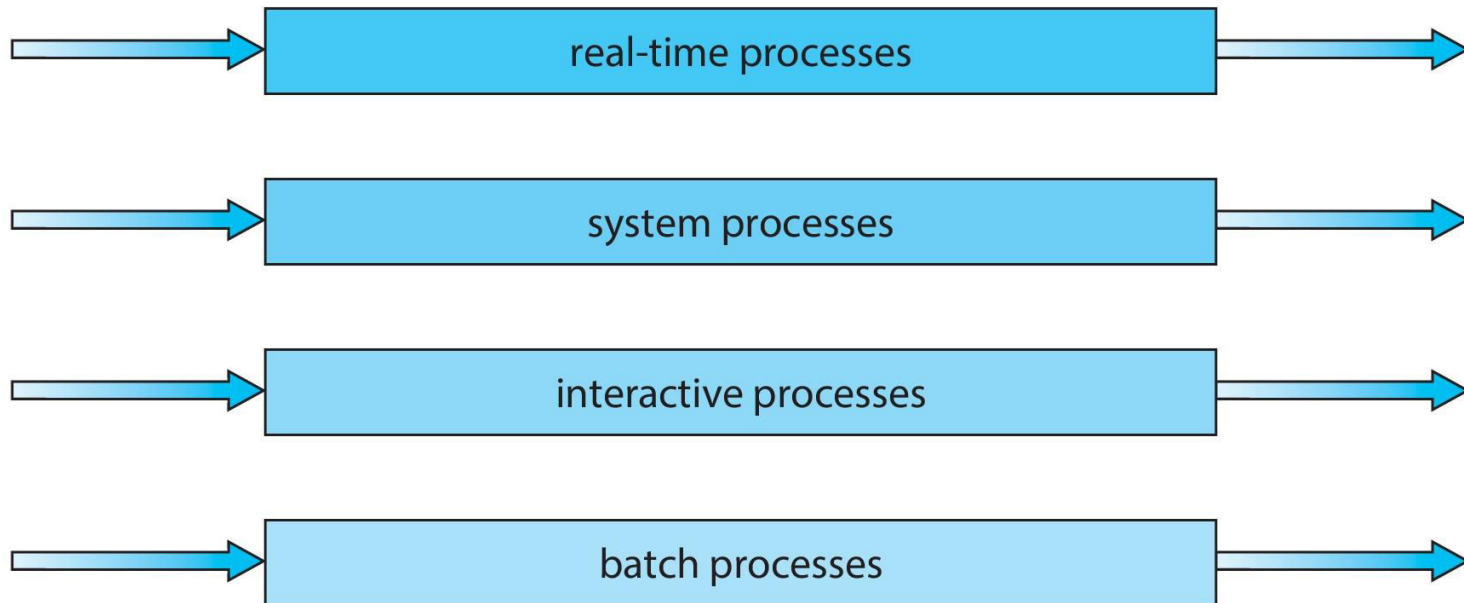
- Bu algoritmada süreçler, farklı gruplara ayrılarak planlaması yapılır.
- En basit haliyle önplandaki (**foreground - interactive**) süreçler bir grupta, arkaplandaki (**background - batch**) süreçler bir grupta tutulur.
- Her iki grup için de farklı planlama yaklaşımı tercih edilebilir.
- Yada bir gruptaki süreçler diğer gruptaki süreçlerden öncelikli olabilir.
- Genellikle Multilevel Queue planlama algoritması hazır süreç kuyruğunu görseldeki gibi farklı kuyruklara böler.



Planlama Algoritmaları

5. Multilevel Queue Scheduling:

highest priority



lowest priority

Planlama Algoritmaları - Multilevel devam...

- Süreçler genellikle **işlem türü**, **işlem önceliği**, **bellek boyutu** gibi özelliklerine göre kuyruklara atanır.
- Önplandaki süreçler **RR**, arkaplandaki süreçler **FCFS** gibi, her kuyruk kendi planlama algoritmasını kullanabilir.
- Ancak kuyruklar arasında sabit öncelikli (**fixed priority**) **preemptive** bir planlama tercih edilebilir.
- Örneğin önplandaki süreçler her zaman arkaplandaki süreçlerden daha yüksek önceliğe sahiptir.
- Başka bir yaklaşım ise kuyruklar arasında CPU planlamanın paylaştırılmasıdır.
- Örneğin arka plan kuyruğu CPU'nun %20'sini FCFS ile planlarken, önplandaki süreçler CPU'nun %80'ini RR ile planlayabilir.

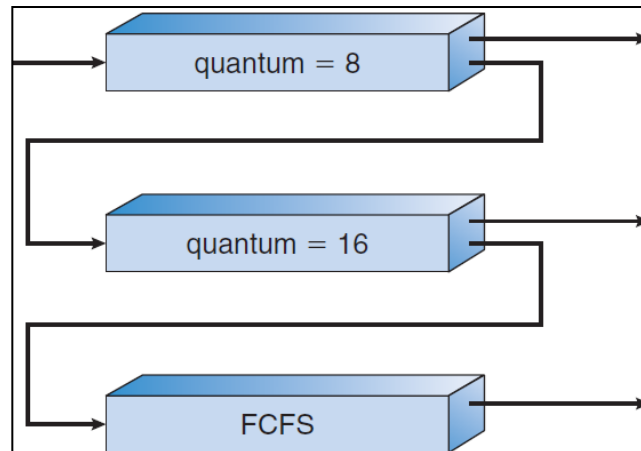
Planlama Algoritmaları

6. Multilevel Feedback Queue Scheduling:

- Multilevel Queue Planlama kullanıldığında süreçler kuyruklara kalıcı olarak alınır. Yani süreçler kuyruk değiştiremez.
- Bu yaklaşım planlamayı kolaylaştırırsa da esnek değildir.
- Bu algoritmada ise süreçlerin kuyruklar arasında geçişine izin verilir.
- Temel fikir, süreçlerin CPU burst zamanlarına göre kuyruklara almaktır.
- Düşük CPU burst zamana sahip süreçler yüksek öncelikli kuyruğa, yüksek CPU burst zamana sahip süreçler düşük öncelikli kuyruğa alınır.
- Düşük öncelikli bir kuyrukta çok uzun süre bekleyen bir süreç daha yüksek öncelikli bir kuyruğa taşınabilir.
- Burada kullanılan yaşlandırma (**aging/boosting**) yaklaşımı süreçlerin açlık (**starvation**) durumuna düşmesini engeller.
- Tüm CPU planlama algoritmalarının bir özeti olarak ifade edilebilir. Ancak en karmaşık olanıdır.

Planlama Algoritmaları - MFQ devam...

- Şekildeki 3 kuyruktan, ilk kuyruk ($q=8$) en yüksek önceliğe, son kuyruk (FCFS) ise en düşük önceliğe sahiptir.
- Öncelikli kuyruklardaki süreçler daha düşük öncelikli kuyrukları keser.
- Bu örnekte hazır duruma geçen bir süreç ilk kuyruğa alınır ve 8 quantum zaman bölmesine sahiptir. Bu sürede bitmezse bir sonraki kuyruğun sonuna eklenir.
- İlk kuyruk boşsa bir sonraki kuyruktaki süreçlere geçilir ve 16 ms quantum kullanılır. Bu sürede tamamlanmayan süreçler son kuyruğa taşınır.
- Ortadaki kuyruk boşsa son kuyruğa geçilir ve FCFS'ye göre planlanır.
- Bu yaklaşımla kısa CPU burst zamanı kullanan süreçler öncelikle tamamlanır.



Thread Planlama

- Kullanıcı düzeyindeki (**user-level**) bir thread'in CPU'da yürütülebilmesi için çekirdek düzeyinde (**kernel-level**) bir thread'le ilişkilendirilmesi gerekir,
- **Many-to-many** ve **many-to-one** modeli kullanan sistemlerde thread planlama, CPU'yu ele geçirmek isteyen thread'ler aynı süreci temsil ettiği için **process contention scope (PCS)** olarak isimlendirilir.
- **One-to-one** model kullanan sistemlerde CPU rekabeti tüm sistem thread'leri için olduğundan **system-contention scope (SCS)** olarak ifade edilir.
- **Window, Linux ve Solaris** gibi sistemlerde **SCS** ile thread'ler planlanır. Sistem CPU planlama algoritmaları kullanılır.
- **PCS** planlama, önceliğe göre yapılır. Programcı user-level thread'lerin öncelik düzeyini belirler.
- **Pthreads** kütüphanesi aşağıdaki planlama kapsamlarını sağlar;

`PTHREAD_SCOPE_PROCESS`: PCS planlama alanı,

`PTHREAD_SCOPE_SYSTEM`: SCS planlama alanı. many-to-many için her bir user thread için bir kernel thread türeterek one-to-one gibi çalışması sağlanır.

Thread Planlama

- **Pthreads** kütüphanesinde thread scope'unu ayarlamak ve öğrenmek için;
`pthread_attr_setscope(pthread_attr_t, int),`
`pthread_attr_getscope(pthread_attr_t *, int *)`
- Bazı sistemler (lin, mac) sadece `PTHREAD_SCOPE_SYSTEM` destekler.
- Yandaki program öncelikle mevcut thread scope'unu öğrenir ve ardından thread scope'unu `PTHREAD_SCOPE_SYSTEM` olarak ayarlar.
- Daha sonra SCS ilkesine dayalı çalışacak 5 tane thread oluşturur.

```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5
void *runner(void *param);
int main(){
    int i, scope;
    pthread_t tid[NUM_THREADS];
    pthread_attr_t attr;

    pthread_attr_init(&attr);
    if (pthread_attr_getscope(&attr, &scope) != 0)
        fprintf(stderr, "Planlama scope'u alınamadı\n");
    else{
        if (scope == PTHREAD_SCOPE_PROCESS)
            printf("PTHREAD_SCOPE_PROCESS\n");
        else if (scope == PTHREAD_SCOPE_SYSTEM)
            printf("PTHREAD_SCOPE_SYSTEM\n");
        else
            fprintf(stderr, "Geçersiz scope.\n");
    }

    pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);

    for (i = 0; i < NUM_THREADS; i++)
        pthread_create(&tid[i], &attr, runner, NULL);

    for (i = 0; i < NUM_THREADS; i++)
        pthread_join(tid[i], NULL);
}

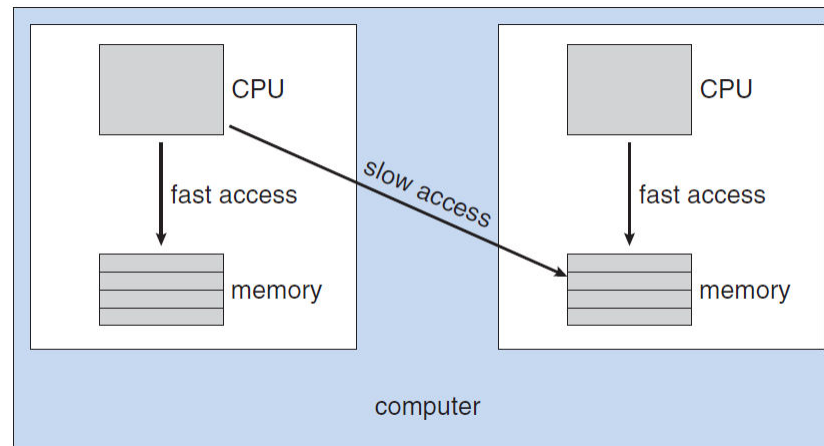
void *runner(void *param){
    pthread_exit(0);
}
```

Multiple-Processor Planlama

- Şimdiye kadar anlatılan CPU planlama modelleri tek CPU'lu sistemler içindir,
- Bir sistemde birden fazla CPU mevcut olduğunda planlama daha karmaşık hale gelir,
- Bu durumda hazır süreç kuyruğundaki süreçler, boşta olan herhangi bir CPU üzerinden planlanabilir.
- Burada CPU'ların homojen olup olmadığı, bazı CPU'ların özel G/Ç sistemlerine bağlı olmasından dolayı bazı süreçlerin bu CPU'ya atanma zorunluluğu olması gibi zorluklar vardır.
- Multiple-Processor planlamadaki bir yaklaşım, tüm planlama kararlarının bir CPU (master) üzerinden yürütülmesidir. Diğer CPU'lar sadece kullanıcı kodlarını yürütür. (**asymmetric multiprocessing**)
- Diğer yaklaşımda (**symmetric multiprocessing - SMP**) ise her işlemci kendini planlar. Tüm süreçler ortak bir hazır süreç kuyruğunda olabilir yada ayrı ayrı tutulabilir.
- Modern işletim sistemleri (Linux, Mac OS X, Windows) SMP kullanır.

Multiple-Processor Planlama

- SMP sistemlerde bir süreç bir işlemcide yürütülmeye başladıktan sonra, sonraki planlamalarda da aynı işlemcide yürütülmek istenmektedir (önbellek için). Bu yaklaşıma işlemci yakınlığı (**processor affinity**) denir.
- Eğer sistem, bir sürecin aynı işlemcide devam etmesini zorlarsa buna **hard affinity**, eğer aynı işlemci garantisi yoksa **soft affinity** olarak isimlendirilir.
- Linux soft affinity yaklaşımını uygular ancak hard affinity özelliğini destekleyen **sched_setaffinity()** sistem çağrısını sunar.
- Bir sistemin ana bellek mimarisi de processor affinity durumunu etkiler. CPU'nun ana belleğin belirli bölümlerine diğer bölümlerden daha hızlı erişmesi, süreçlerin ana belleğe yüklenirken bu bölüme yüklenmesini gerektirir.

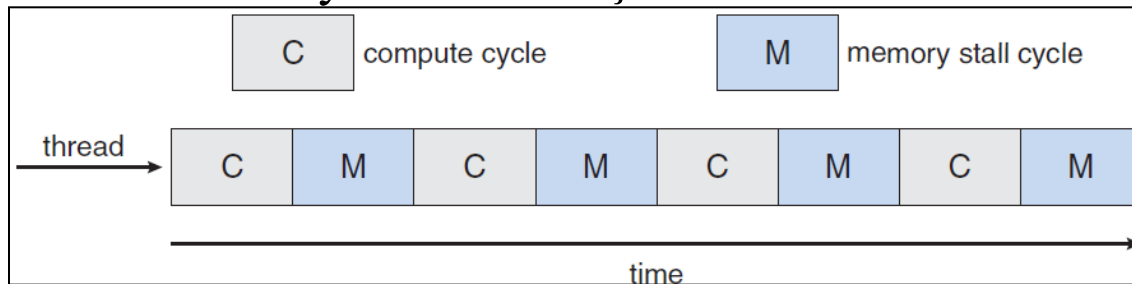


Multiple-Processor Planlama

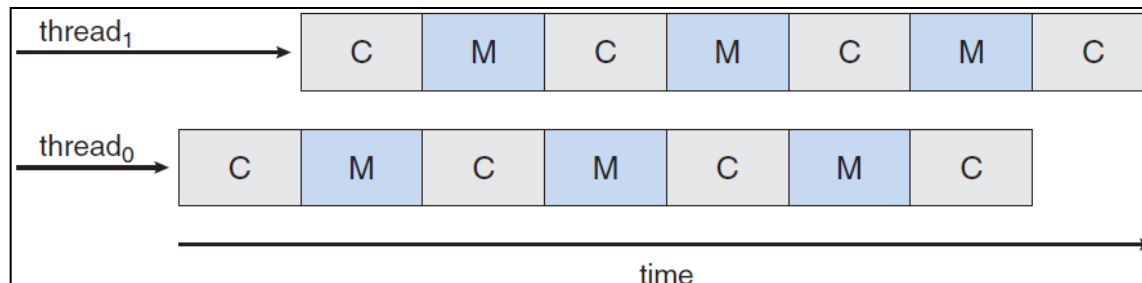
- SMP sistemlerde birden fazla işlemciye sahip olmanın faydalarını tam olarak görmek için iş yükünün işlemciler arasında paylaşımını (**load balancing**) gerektirir.
- Load balancing, hazır süreç kuyruğunun tüm işlemciler için ortak tutulduğu sistemlerde gereksizdir.
- Çoğu modern SMP sistemlerde her işlemci için özel hazır süreç kuyruğu bulunur.
- Load balancing, **push migration** ve **pull migration** olarak iki genel yaklaşıma sahiptir.
- **Push migration** ile bir görev tüm işlemcilerdeki süreçleri denetler ve dengesizlik bulunduğunda aşırı yüklenmiş işlemciye yükü boşta yada daha az meşgul olan işlemciye taşır (push-iter).
- **Pull migration** ile boşta bekleyen işlemci meşgul işlemcilerden süreci çeker.
- Linux ve FreeBSD planlayıcıda iki teknik de kullanılır.
- Load balancing, processor affinity yaklaşımıyla **tezatlık** oluştur. Bu nedenle bazı sistemlerde yük dengesizliği belli bir eşik değerine ulaşırsa taşıma yapılır.

Multiple-Processor Planlama

- SMP sistemlerin çoğu günümüzde **multiple-processor** yerine **multicore** (çok çekirdekli) sistemler olarak üretilir.
- Multicore sistemlerde planlama, 1 core'da birden fazla thread olmasıyla zorlaşır.
- Çekirdeklerin birden fazla thread donanımına sahip olmasının nedeni bellek erişimlerindeki yavaşlığın sonucu oluşan beklemleri (**memory stall**) gidermektir.
- Aşağıdaki şekilde thread 0 yürütülürken çekirdek %50 oranında belleği bekler.



- Bir core'da 2 thread donanımı olan bir sistemde ise;

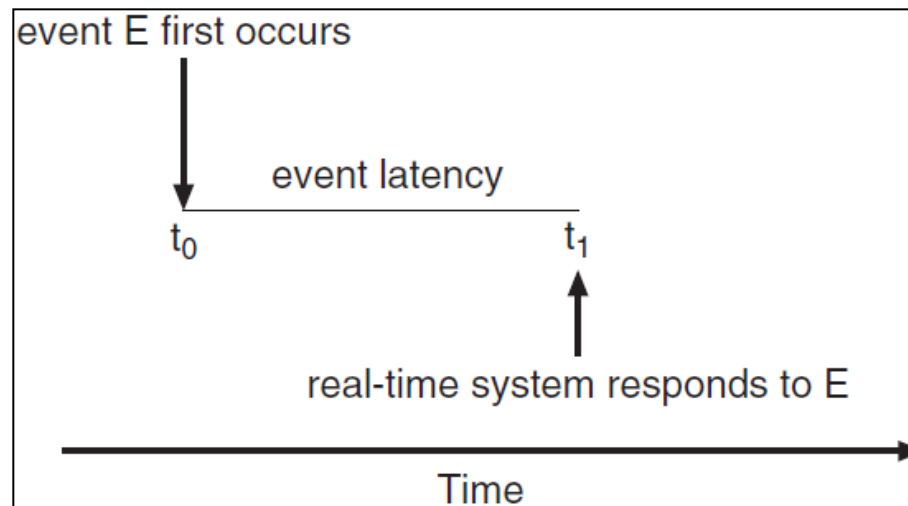


Multiple-Processor Planlama

- Genel olarak, bir işlemci çekirdeğinin thread'lerini yönetmenin iki yolu vardır.
- **coarse-grained**: Bu yöntemde thread'ler arası geçiş, yürütülen thread'de bir bellek beklemesi gibi uzun bir gecikme oluştuğunda yapılır.
- **fine-grained**: Bu yöntemde ise çok daha hızlı thread geçişleri gerçekleşir. Bu sistemlerde thread switching tasarımı yer aldığı için fazla içerik değişim verimsizliği ortadan kaldırılır.
- Sonuç olarak, çok çekirdekli SMP sistemleri **iki seviyeli** bir CPU planlama düzeyine sahip olmalıdır.
- **Birinci düzeyde** her donanım çekirdeğinde hangi iş parçacıklarının yürütüleceğinin planlaması yapılır.
 - Bu düzeyde planlamayı işletim sistemi algoritmaları yürütür.
- **İkinci düzeyde** her bir çekirdek içerisindeki donanım thread'lerinin planlanmasıdır.
 - Burada işlemci üreticileri (Ultrasparc) roundrobin gibi yada Intel'de kullanılan öncelik algoritması gibi farklı teknikler kullanabilirler.

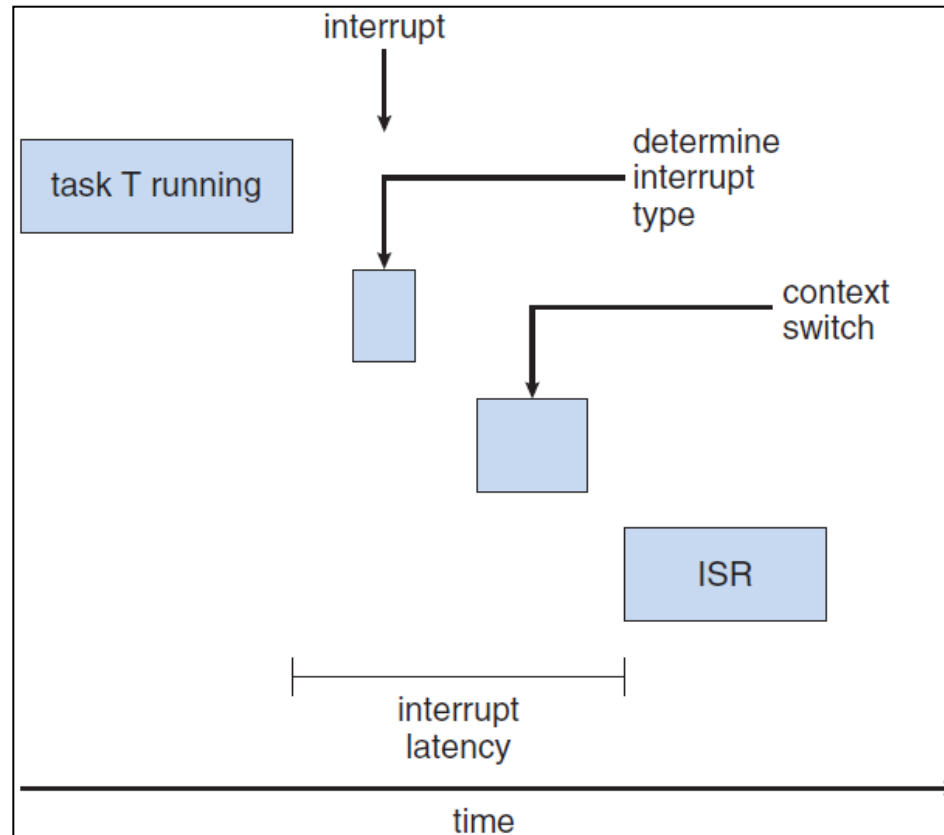
Gerçek Zamanlı CPU Planlama

- Gerçek zamanlı işletim sistemlerinde CPU planlama iki şekilde ele alınır.
- **Soft real-time systems:** Kritik bir gerçek zamanlı sürecin planlanma garantisi yoktur. Sadece kritik olmayan süreçlerden önce planlanacağı bilinir.
- **Hard real-time systems:** Planlama sıkıdır. Yani bir süreç deadline zamanı içerisinde planlanmalıdır.
- Gerçek zamanlı sistemlerde gecikme (**latency**), bir olay meydana geldiğinde sistem tepki süresi olarak belirtilebilir.
- Her sistemin tolere edebileceği gecikme değeri farklı olabilir.



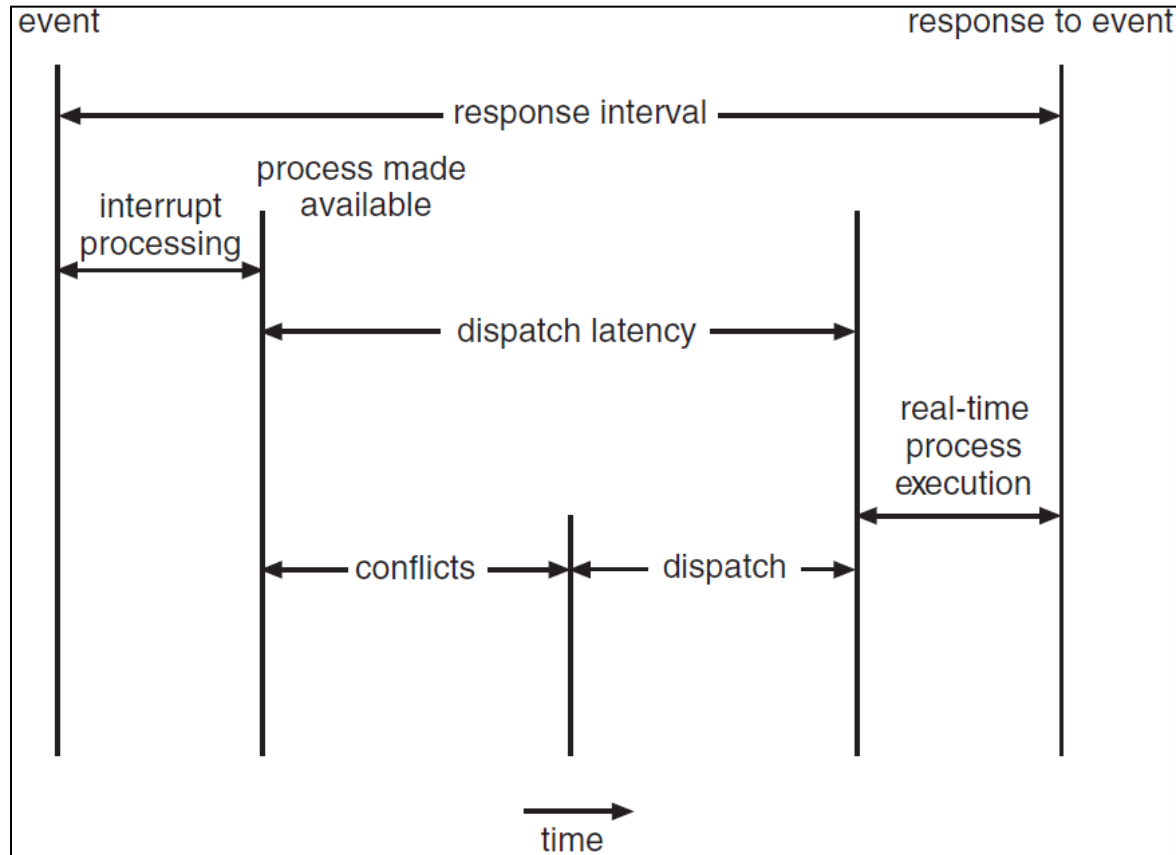
Gerçek Zamanlı CPU Planlama

- Gerçek zamanlı sistemlerde gecikmenin sebebi iki nedenle olabilir;
- Interrupt latency**, kesme tipinin belirlenmesi ve context switch sürecinde geçen zamanı tanımlar. Kesmenin oluşmasından Interrupt Service Routine (ISR), süresince geçen toplam süredir.



Gerçek Zamanlı CPU Planlama

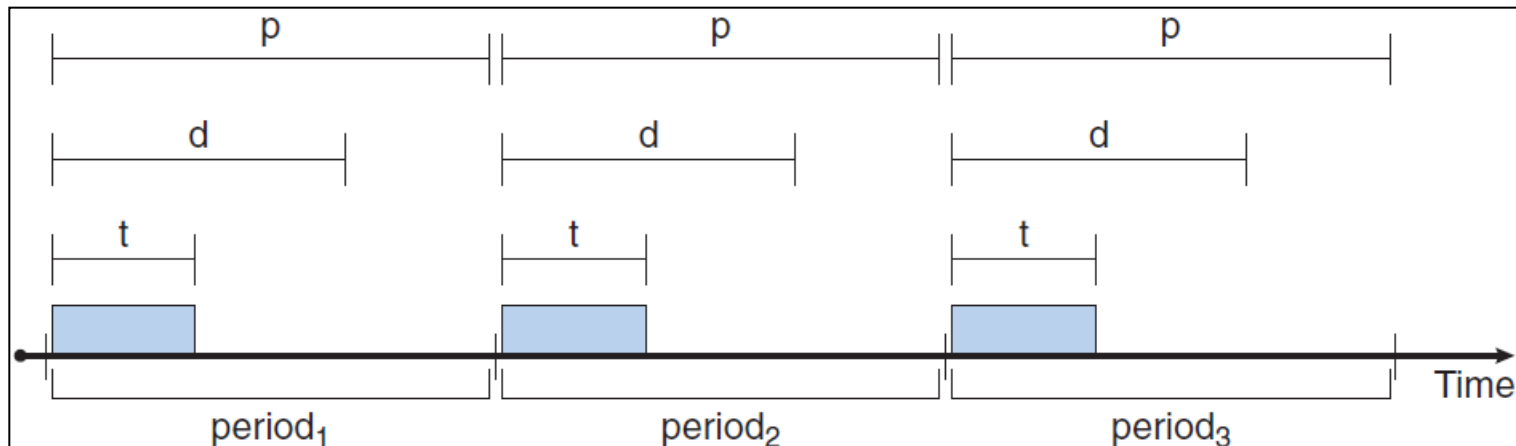
- **Dispatch latency**, planlama zamanlayıcısının bir süreci durdurup diğer bir süreci başlatması için geçen zamanı ifade eder. **Conflict** durumu çekirdekte yürüyen bir sürecin önlenmesi ve düşük öncelikli bir süreç tarafından kullanılan kaynağın serbest bırakılması esnasında harcanan zamandır.



Gerçek Zamanlı CPU Planlama

1. Priority-Based Planlama: Gerçek zamanlı sistemlerde planlama, öncelik tabanlı bir **preemptive** algoritmayı desteklemek zorundadır.

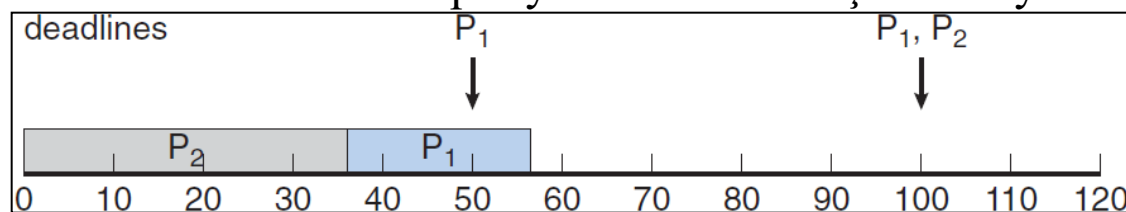
- Gerçek zamanlı sistemlerde bir süreç periyodik (**periodic**) olarak kabul edilir.
- Periyodik bir işlem CPU'yu tuttuğunda işlemciyi (**t**) süresince kullanılır. CPU tarafından tamamlanması gereken bir son tarih (**d**) ve bir periyot (**p**) zamanı vardır.
- İşlem zamanı, son tarih ve periyot arasındaki ilişki $0 \leq t \leq d \leq p$ olarak ifade edilir.
- Planlayıcılar, kabul denetimi (**admission-control**) algoritmasıyla, sürecin son tarih zamanında tamamlanabileceğini belirler ve **kabul eder** yada ilgili son tarih zamanında sürecin tamamlanamayacağından **reddeder**.



Gerçek Zamanlı CPU Planlama

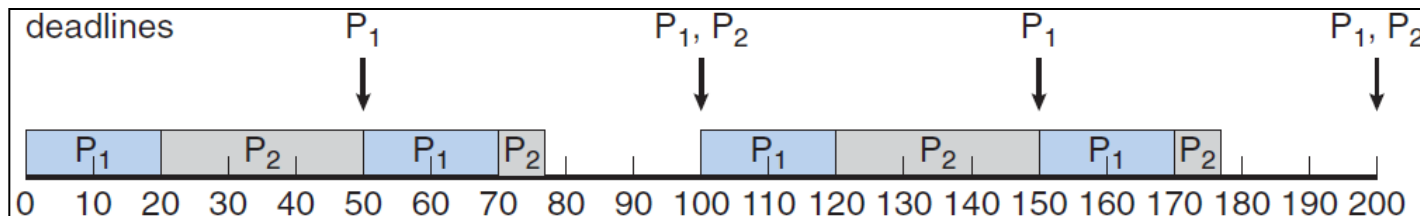
2. Rate-Monotonic Planlama: Bu algoritma, CPU periyodu düşük olan süreçlere daha yüksek öncelik atar. Bir sürecin periyodu ne kadar düşüğe önceliği o kadar yüksek olur. Bu yaklaşımdaki temel mantık CPU ihtiyacı daha sık olan görevlere daha yüksek öncelik vermektir.

- Örneğin P1 ve P2 gibi iki süreç için periyotlar **p1=50** ve **p2=100**, işlem süreleri **t1=20** ve **t2=35** olsun.
- Burada her süreç için **deadline**, bir sonraki periyodun başlangıcına kadar CPU burst'ünü tamamlamasını gerektirir.
- CPU utilizasyonuna baktığımızda $P1 \Rightarrow 20/50 = \%40$, $P2 = 35/100 = \%35$ ve toplam $\%75$. Bu durumda süreçlerin CPU'nun mevcut çevrimleriyle planlanabileceği öngörülür.
- Bu örnekte P2'nin P1'den daha yüksek olduğu kabul edilseydi;
- P2 önce işletilecek, 35'de tamamlanacak, sonra P1 yürütülecek ve 55'de tamamlanacaktır. Ancak P1'in periyodu 50 olduğu için periyodu kaçıracak ve 50. zamanda P1 deadline'ı kaçıracaktı. Böylece CPU bir sonraki P2 periyoduna kadar boşa bekleyecektir.



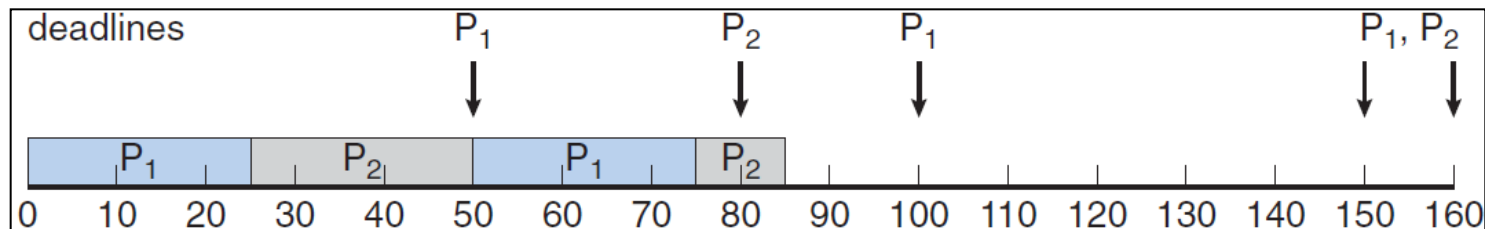
Gerçek Zamanlı CPU Planlama -RM devam...

- Bu algoritmaya göre P1'in periyodu düşük olduğu için P1, P2'den daha yüksek önceliğe sahip olacaktır.
- Burada, P1 öncelikle yürütülmeye başlar ve 20. zamanda CPU burst'unü tamamlar. Böylece deadline'ını karşılar.
- Bu noktada P2 çalışmaya başlar ve 50. zamana kadar çalışır.
- Bu anda P2'nin tamamlanmasına 5 birim zaman olsa da P1 tarafından önlenir.
- P1 devreye girer ve 70. zamanda çalışma zamanını tamamlar.
- Ardından 75. birim zamana kadar P2 ilk periyodunda tamamlayamadığı 5 birim zamanlık kısmını tamamlar.
- Sistem, 100. zamanda P1 ve P2'yi yeniden planlayıncaya kadar boşa bekleyecektir.



Gerçek Zamanlı CPU Planlama -RM devam...

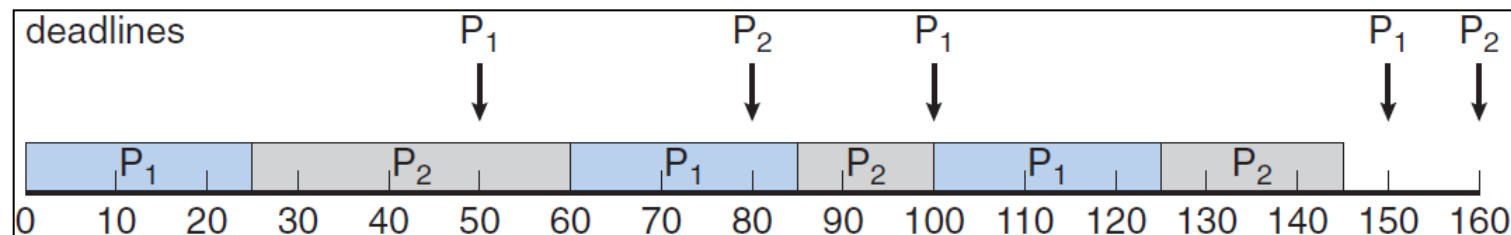
- Örneğin; P1 ve P2 süreçleri için; **p1=50** ve **t1=25**, **p2=80** ve **t2=35** olsun.
- P1'in periyodu düşük olduğu için P1 yüksek önceliklidir.
- CPU utilizasyon, $(25/50) + (35/80) = 0.94$ (iki süreç planlanabilir görünüyor)
- Başlangıçta P1, 25. zamana kadar CPU burst'ünü tamamlar. Sonra P2 yürütülür.
- P2 50. zamanda tamamlanmasına 10 birim zaman kala P1 tarafından önlenir.
- P1 50. zamandan 75. zamana kadar tamamlanır. Ardından P2 75'ten 85. birim zamana kadar kalan işlem zamanını tamamlar. Ancak P2, 80. zamanda deadline'ını kaçırmıştır.
- Bu nedenle bu algoritma süreç sayısı arttıkça CPU utilizasyonu %69 civarında karşılayabilmektedir.
- İki süreç için bu oran %83 civarındadır. Son örnekte %96 olan utilizasyon istenilen deadline için planlanamamıştır.



Gerçek Zamanlı CPU Planlama

3. Earliest-Deadline-First (EDF) Planlama: Deadline süresi en kısa olan süreç en yüksek önceliğe sahiptir. Bir süreç yürütülebilir hale geldiğinde sisteme deadline'ını bildirmek zorundadır.

- Bir önceki örnekteki gibi; P1 ve P2 süreçleri için; **p1=50 ve t1=25**, **p2=80 ve t2=35** olsun.
- P1 süreci en düşük deadline'e sahiptir ve öncelikle yürütülür. Ardından P2 devreye girer.
- P2'nin yürütümü 50. zamanda P1 tarafından önlenebilir. Çünkü bir sonraki en yakın deadline P2 için 80, P1 için 100 olduğundan artık P2, P1'den daha yüksek önceliğe sahiptir.
- P1 süreci tekrar 60. zamanda çalışmaya başlar ve 85. zamanda ikinci burst'ünü tamamlar.
- 85. zamanda P2 devreye girer ancak 100. zamanda P1'in periyodu daha düşük olduğundan ($150 < 160$) P2, P1 tarafından önlenir.
- P1 üçüncü burst'ünü 125. zamanda tamamlar ve P2'nin kalan 20 birim zamanlık kısmı 145. zamanda tamamlanır. CPU bir sonraki periyot olan 150. zamana kadar bekler.



Gerçek Zamanlı CPU Planlama

4. Proportional Share Planlama: Oran paylaşım algoritması olarak tanımlanabilen bu yapı CPU kaynağı olan T birimi süreçler arasında paylaştırır.

- Örneğin CPU'nun A, B ve C gibi üç süreç arasında paylaştırılacağını düşünelim.
- Toplam payın $T=100$ olduğu bir ortamda, $A=50$, $B=15$ ve $C=20$ paya sahip olsun.
- Bu durumda A kaynağın %50, B %15 ve C'de %20 payına sahip olur.
- Orantılı paylaşım planlayıcıları kabul kontrol (**admission-control**) yapısına sahip olmalıdır.
- Kabul kontrol yapıları, bir istek geldiğinde ilgili isteğe yetecek bir pay varsa isteği kabul eder.
- Bu örnekte örneğin 30 pay isteyen bir D sürece geldiğinde, kalan pay 15 birim olduğu için istek kabul edilmeyecektir.

Gerçek Zamanlı CPU Planlama

5. POSIX Real-Time Planlama: POSIX standardı gerçek zamanlı hesaplama için POSIX 1b. uzantısı sağlamaktadır.

- POSIX gerçek zamanlı threadleri planlamak için iki planlama sınıfı sunar.
 - **SCHED-FIFO**
 - **SCHED_RR**
- **SCHED_FIFO**, FCFS tabanlı çalışan bir kuyruk yapısıdır. Ancak eşit öncelikli thread'ler arasında zaman paylaşımı yapmaz. En yüksek öncelikli thread'in çalıştırılması garanti edilir.
- **SCHED_RR**, round-robin algoritması kullanır. Eşit öncelikli thread'ler arasında zaman bölme sağlar.
- POSIX APIi planlama kurallarını get/set için iki fonksiyon sunar.
 - `pthread_attr_getsched_policy(pthread_attr_t *attr, int *policy)`
 - `pthread_attr_setsched_policy(pthread_attr_t *attr, int policy)`
- Burada ilk parametre thread attribütünü, ikinci parametre planlama kuralını tutar.

Gerçek Zamanlı CPU Planlama

5. POSIX Real-Time Planlama:

```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5
void *runner(void *param);
int main(int argc, char *argv[]){
    int i, policy;
    pthread_t tid[NUM_THREADS];
    pthread_attr_t attr;

    pthread_attr_init(&attr);

    if (pthread_attr_getschedpolicy(&attr, &policy) != 0)
        fprintf(stderr, "policy alınamıyor.\n");
    else{
        if (policy == SCHED_OTHER) printf("SCHED_OTHER\n");
        else if (policy == SCHED_RR) printf("SCHED_RR\n");
        else if (policy == SCHED_FIFO) printf("SCHED_FIFO\n");
    }

    if (pthread_attr_setschedpolicy(&attr, SCHED_OTHER) != 0)
        fprintf(stderr, "policy ayarlanamadı.\n");

    for (i = 0; i < NUM_THREADS; i++)
        pthread_create(&tid[i], &attr, runner, NULL);

    for (i = 0; i < NUM_THREADS; i++)
        pthread_join(tid[i], NULL);
}

void *runner(void *param){
    pthread_exit(0);
}
```

İşletim Sistemi Örnekleri - Linux

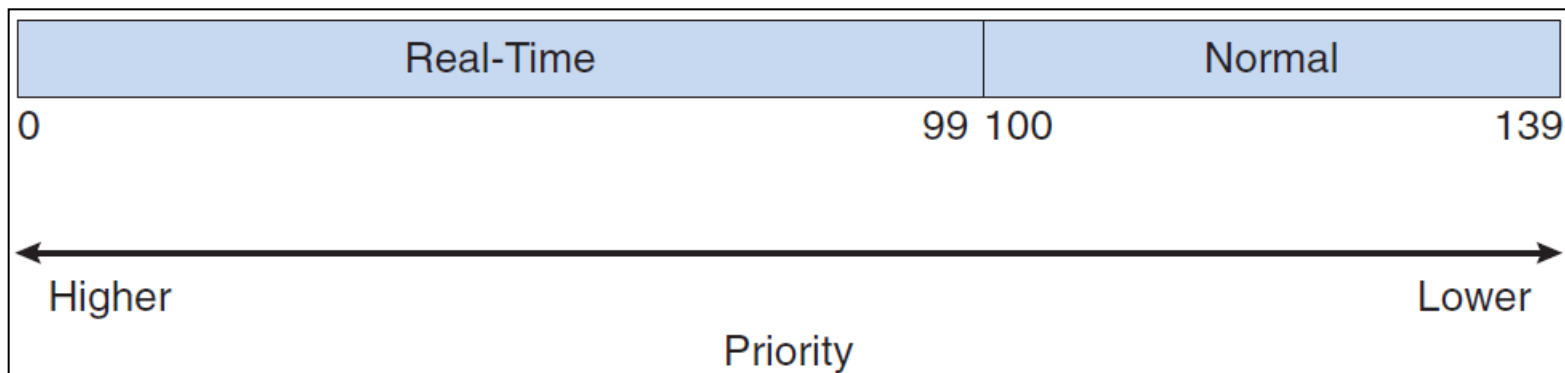
- **Linux**, işletim sisteminde CPU planlama 2.5 sürümüne kadar geleneksel UNIX planlama algoritmasını kullanır.
- Ancak bu algoritma SMP sistemler için uygun değildir.
- Çekirdek 2.5 ile birlikte $O(1)$ olarak isimlendirilen planlama algoritması geliştirildi
- Bu algoritma SMP (CPU benzerliği, yük dengeleme vb.) için uygun olsa da masaüstü sistemler için artan etkileşimli süreçler için yeterli olmadı.
- Son olarak çekirdek 2.6 ile birlikte **Completely Fair Scheduler (CFS)** olarak isimlendirilen algoritma geliştirildi.
- Linux, planlama sınıfı (**scheduling classes**) yaklaşımı kullanır.
- Bir sunucu sistem planlama algoritması ile bir mobil sisteminki farklı olabilir.
- Standart linux çekirdeği; (1) Varsayılan CFS planlama, (2) Gerçek zamanlı planlama algoritması sunar.

İşletim Sistemi Örnekleri - Linux

- **CFS** planlama, her sürece, CPU işlem süresi için $-20 + 19$ arasında bir değer atar.
- Bu değerlerden düşük değere sahip süreç daha yüksek önceliğe sahiptir.
- Bir sürecin varsayılan önceliği 0'dır.
- CFS planlama öncelikleri doğrudan atamaz. Her süreç için, sanal çalışma zamanı (vruntime) verisi tutar.
- **vruntime**, her sürecin önceliğe dayanan bir bozulma faktörü tanımlanır.
- Normal önceliğe sahip süreç (değeri 0), 200 ms çalışırsa, vruntime 200 ms olur,
- Düşük önceliğe sahip bir süreç, 200 ms çalışırsa, vruntime 200 ms'den büyük olur,
- Yüksek öncelikli bir süreç, 200 ms çalışırsa, vruntime 200 ms'den küçük olur.
- Bundan sonra planlayıcı vruntime zamanı düşük olan süreci seçer.
- Aynı öncelik değerine sahip iki süreç için, birinci süreç CPU'ya bağlı (yüksek CPU-bound), ikinci süreç G/Ç'a bağlı (yüksek I/O-bound) olsun.
- Bu durumda I/O bound'u büyük olan süreç, daha kısa sürelerle CPU'da çalıştırılacağı için CPU-bound'u büyük olan süreçten daha yüksek öncelikli olur.

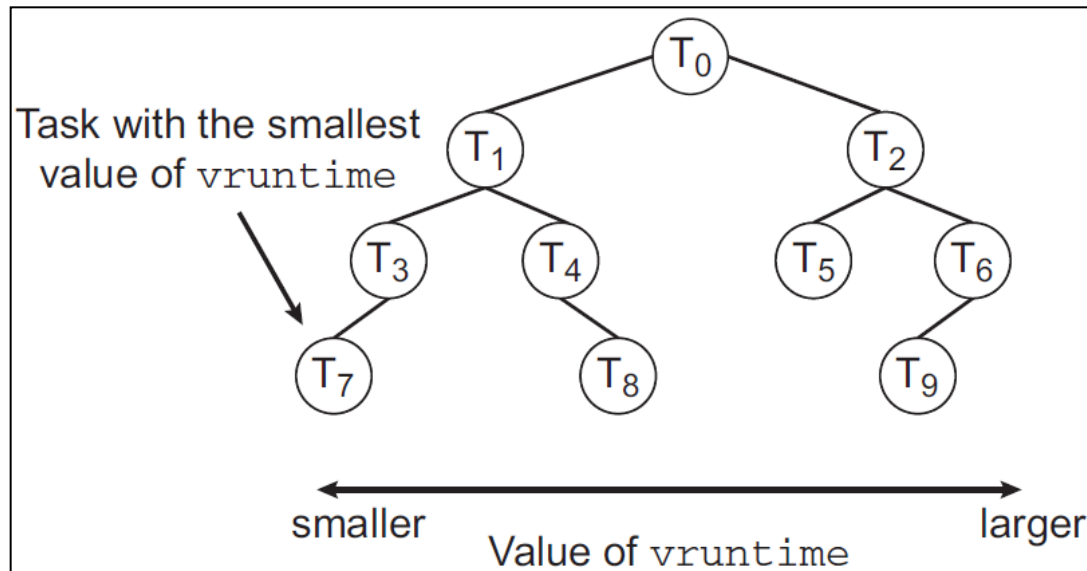
İşletim Sistemi Örnekleri - Linux

- Linux ayrıca gerçek zamanlı planlama desteği sunar. (SCHED_FIFO, SCHED_RR)
- Linux gerçek zamanlı süreçler için birincil (0-99) ve normal süreçler için ikincil (100-139) öncelik aralığı tanımlar.
- Gerçek zamanlı görevlerin önceliği statiktir ve düşük olan değer yüksek önceliğe sahiptir.
- Normal süreçlerin -20+19 aralığındaki göreceli öncelikleri 100-139 arasındaki değerlerle eşlenir. (-20 değeri 100 öncelikli, +19 değeri 139)



İşletim Sistemi Örnekleri - Linux

- **CPS planlama**, çalıştırılacak bir sonraki süreci belirlemek için **red-black tree** olarak adlandırılan bir ağaç algoritması kullanır. (runtime temelinde)
- Bir süreç çalıştırılabilir olduğunda ağaca eklenir. **Düşük runtime** değerine sahip görevler ağacın **sol tarafına**, yüksek öncelikli görevler **sağ tarafına** eklenir.
- **En soldaki düğüm en düşük anahtar** değerine sahiptir. (**En yüksek öncelikli**)
- Burada en düşük önceliği belirleme performansı $O(Lg N)$ 'dir (N=süreç sayısı)
- Linux performans için bu değeri **rb_leftmost** ile ön belleğe alır.



İşletim Sistemi Örnekleri - Windows

- **Windows**, öncelik tabanlı preemptive bir süreç planlama algoritması kullanır.
- Windows en yüksek öncelikli thread'in her zaman yürütümünü sağlar.
- Planlamanın işlendiği Windows çekirdek bölümü **dispatcher** olarak adlandırılır.
- Dispatcher tarafından seçilen bir thread, daha yüksek öncelikli bir thread tarafından önleninceye, sonlanıncaya, quantum süresi bitinceye yada bir sistem çağrısıncaya (I/O) bloklanıncaya kadar sürdürülecektir.
- Dispatcher, 1-15 arası değişken sınıf (**variable class**), 16-31 arası gerçek zamanlı sınıf (**real-time class**) olmak üzere önceliklendirme için 32 seviye kullanır. (0, bellek yönetimi için kullanılan thread)
- Dispatcher, her öncelik planı için bir kuyruk tanımlar ve bu kuyrukları hazır süreç bulmak için yüksekten düşüğe doğru tarar.
- Hazır süreç yoksa, **idle** olarak bilinen özel bir thread'i devreye sokar.
- Windows çekirdeğindeki öncelik numaraları ile **Windows API**'i arasında **6 öncelik sınıfının** tanımlandığı bir ilişki vardır.

İşletim Sistemi Örnekleri - Windows

- Bu öncelik sınıfları;
 - `IDLE_PRIORITY_CLASS`, `BELOW_NORMAL_PRIORITY_CLASS`, `NORMAL_PRIORITY_CLASS`, `ABOVE_NORMAL_PRIORITY_CLASS`, `HIGH_PRIORITY_CLASS`, `REALTIME_PRIORITY_CLASS`'dir.
- Bir sürecin varsayılan önceliği `NORMAL_PRIORITY_CLASS` ile tanımlanır.
- Windows API'i, bir sürecin önceliğini değiştirmek için **SetPriorityClass()** sunar.
- `REALTIME_PRIORITY_CLASS` sınıfı dışındaki süreç öncelikleri değişebilir.
- Belirli bir sınıfın içindeki bir thread ile aynı sınıfdaki diğer thread'ler arasında da bir göreceli öncelik değeri vardır.
- Bu görece öncelikler;
 - `IDLE`, `LOWEST`, `BELOW_NORMAL`, `NORMAL`, `ABOVE_NORMAL`, `HIGHEST`, `TIME_CRITICAL`'dir

İşletim Sistemi Örnekleri - Windows

- Bir thread'in önceliği hem ait olduğu sınıfa, hem de bu sınıf içerisindeki göreceli öncelik değerine dayanır.
- Örneğin sınıfı above normal olan bir thread, göreceli önceliği normal ise öncelik değeri 10'dur.
- Ayrıca her sınıfın da temsil ettiği bir öncelik değeri vardır ve bu değer NORMAL görece öncelik değeri ile tanımlanır. Örn: NORMAL_PRIORITY_CLASS = 8'dir
- Bir thread'in önceliği SetThreadPriority() ile değiştirilse de başlangıçta ait olduğu sürecin önceliğine sahiptir.

	real-time	high	above normal	normal	below normal	idle priority
time-critical	31	15	15	15	15	15
highest	26	15	12	10	8	6
above normal	25	14	11	9	7	5
normal	24	13	10	8	6	4
below normal	23	12	9	7	5	3
lowest	22	11	8	6	4	2
idle	16	1	1	1	1	1

İşletim Sistemi Örnekleri - Windows

- Bir thread, quantum zamanı dolduğunda kesilir. Thread **variable-class** sınıfında ise önceliği azaltılır. Ancak ait olduğu sürecin önceliğinin altına inemez.
- Ayrıca variable-class'da olan bir thread, bekleme durumundan hazır duruma geçtiğinde önceliği artırılır.
- Artırım miktarı beklediği aygıta bağlıdır. Klavye için daha yüksek artırım, disk için daha düşük artırım.
- Böylece fare ve klavye bekleyen iş parçacıkları için daha iyi bir performans sağlanır.
- Ayrıca Windows, NORMAL öncelik sınıfındaki süreçler arasında ön planda (**foreground process**) yürüyen sürece, arkaplanda (**background processes**) yürüyen süreçlere göre daha fazla quantum zamanı verir. (Örneğin 3 birim)

Algoritma Değerlendirme

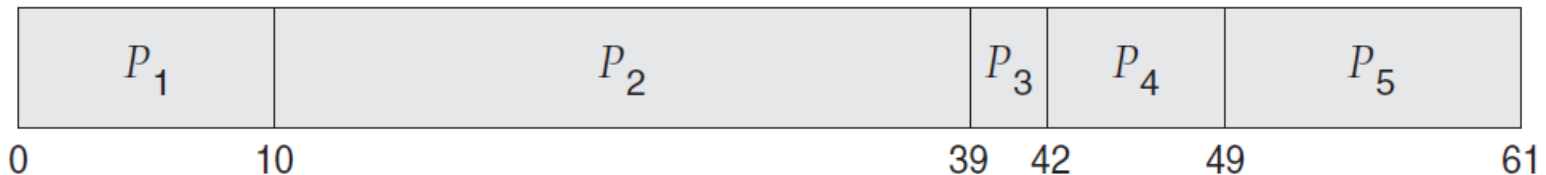
- Belirli bir sistem için hangi CPU planlama algoritmasının uygun olduğunun belirlenmesi önemlidir.
- Bir algoritmayı seçmek için CPU kullanımı (utilization), yanıt zamanı (response time), iş hacmi (throughput) gibi parametreler tanımlanır.
- Bu ölçümdeki kistaslar aşağıdaki gibi kriterleri hedefleyebilir.
 - Maksimum yanıt süresinin 1 sn. olduğu durumda, CPU kullanımını maksimuma çıkarmak
 - Ortalama dönüş zamanının (turnaround time) toplam çalışma zamanı için lineer olacak şekilde iş hacmini (throughput) maksimuma çıkarmak.
- Seçim kriteri belirlendikten sonra ilgili algoritmayı değerlendirmek için farklı yöntemler kullanılabilir.

Deterministik değerlendirme

- Analitik değerlendirme yöntemidir. Bu yöntem verilen algoritmayı değerlendirmek için formüller aracılığıyla bir sonuç değeri üretir.
- Örneğin aşağıdaki iş yükü altında çalışan bir sistem için (süreçlerin varış zamanları 0. saniye olsun);

Process	Burst Time
P_1	10
P_2	29
P_3	3
P_4	7
P_5	12

- FCFS, SJF ve RR ($q=10$) için hangisi minimum bekleme zamanı verecektir?
- **FCFS için waiting time'ler;**
 - $P_1=0, P_2=10, P_3=39, P_4=42, P_5=49$ ms.'dir.
 - O halde Ortalama Bekleme Zamanı $\Rightarrow (0 + 10 + 39 + 42 + 49)/5 = 28$ ms'dir.

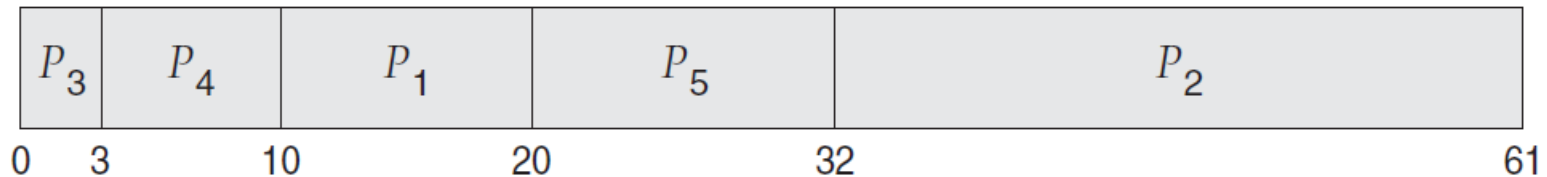


Deterministik değerlendirme

- **SJF (nonpreemptive) için waiting time'ler;**

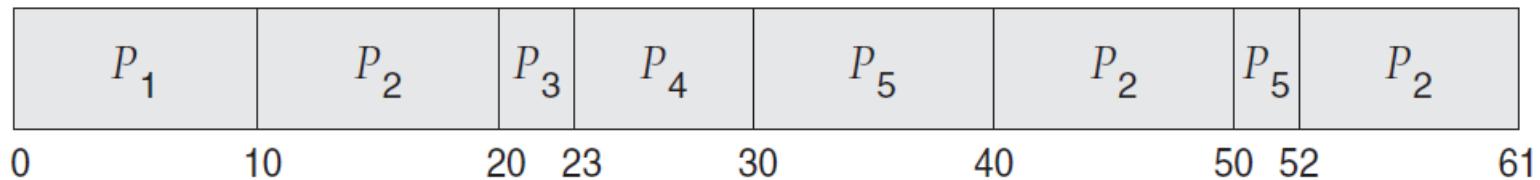
- $P_1=10, P_2=32, P_3=0, P_4=3, P_5=20$ ms.'dir.
- O halde Ortalama Bekleme Zamanı $\Rightarrow (10 + 32 + 0 + 3 + 20)/5 = 13$ ms'dir.

Process	Burst Time
P_1	10
P_2	29
P_3	3
P_4	7
P_5	12



- **RR (q=10) için waiting time'ler;**

- $P_1=0, P_2=32, P_3=20, P_4=23, P_5=40$ ms.'dir.
- O halde Ortalama Bekleme Zamanı $\Rightarrow (0 + 32 + 20 + 23 + 40)/5 = 23$ ms'dir.



- Bu durumda SJF algoritması en düşük ortalama bekleme zamanı sunmaktadır.

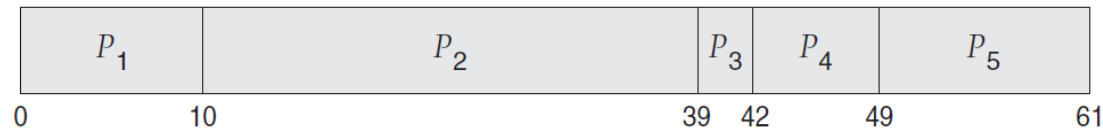
Deterministik değerlendirme

Aynı örneği dönüş süresi (**turnaround time**) için işletelim

- FCFS için; (tüm süreçlerin varış zamanı = 0)

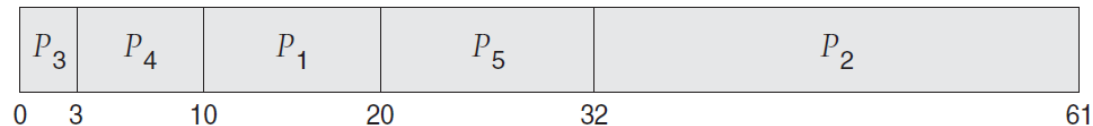
- $P_1=10, P_2=39, P_3=42, P_4=49, P_5=61$ ms.'dir.
- Ortalama turnaround $\Rightarrow (10 + 39 + 42 + 49 + 61)/5 = 40.2$ ms'dir.

Process	Burst Time
P_1	10
P_2	29
P_3	3
P_4	7
P_5	12



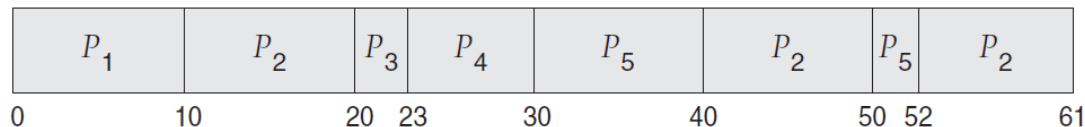
- SJF için;

- $P_1=20, P_2=61, P_3=3, P_4=10, P_5=32$ ms.'dir.
- turnaround $\Rightarrow (20 + 61 + 3 + 10 + 32)/5 = 29.2$ ms'dir.



- RR için;

- $P_1=10, P_2=61, P_3=23, P_4=30, P_5=52$ ms.'dir.
- Ortalama turnaround $\Rightarrow (10 + 61 + 23 + 30 + 52)/5 = 35.2$ ms'dir.



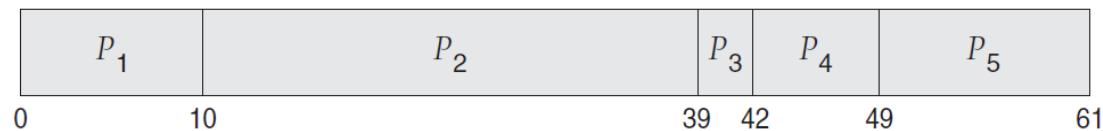
Deterministik değerlendirme

Aynı örneği cevap süresi (**response time**) için işletelim

- FCFS için; (tüm süreçlerin varış zamanı = 0)

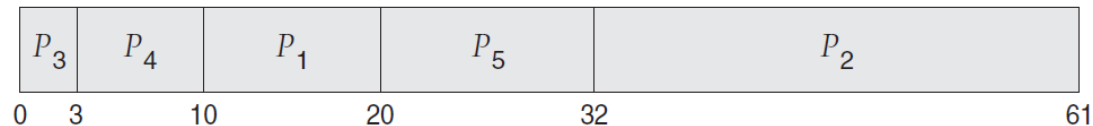
- $P_1=0, P_2=10, P_3=39, P_4=42, P_5=49$ ms.'dir.
 - Ortalama response $\Rightarrow (0 + 10 + 39 + 42 + 49)/5 = 28$ ms'dir.

Process	Burst Time
P_1	10
P_2	29
P_3	3
P_4	7
P_5	12



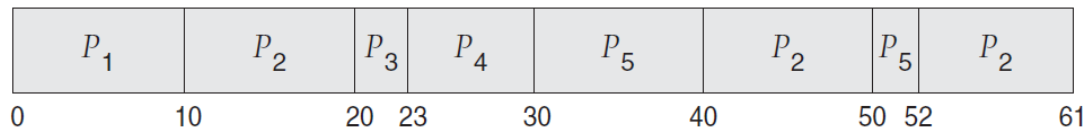
- SJF için;

- $P_1=10, P_2=32, P_3=0, P_4=3, P_5=20$ ms.'dir.
 - response $\Rightarrow (10 + 32 + 0 + 3 + 20)/5 = 13$ ms'dir.



- RR için;

- $P_1=0, P_2=10, P_3=20, P_4=23, P_5=30$ ms.'dir.
 - Ortalama response $\Rightarrow (0 + 10 + 20 + 23 + 30)/5 = 16.6$ ms'dir.



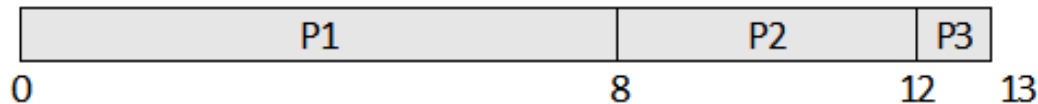
Deterministik değerlendirme

ÖRNEK: Yandaki süreç kümesi için, nonpreemptive bir planlama yaklaşımı ile;

Process	Arrival Time	Burst Time
P_1	0.0	8
P_2	0.4	4
P_3	1.0	1

• FCFS için ortalama dönüş süresi nedir?

- $P_1=8$, P_2 için $12-0.4=11.6$, P_3 için $13-1=12$ ms.'dir.
- Ortalama turnaround $\Rightarrow (8 + 11.6 + 12) / 3 = 10.53$ ms'dir.



• SJF için ortalama dönüş süresi nedir?

- $P_1=8$, $P_2=13-0.4=12.6$ ve $P_3=9-1=8$ ms.'dir.
- turnaround $\Rightarrow (8 + 12.6 + 8) / 3 = 9.53$ ms'dir.



- Preemptive olsaydı turnaround $\Rightarrow (13 + 5 + 1) / 3 = 6.33$ ms. olurdu

